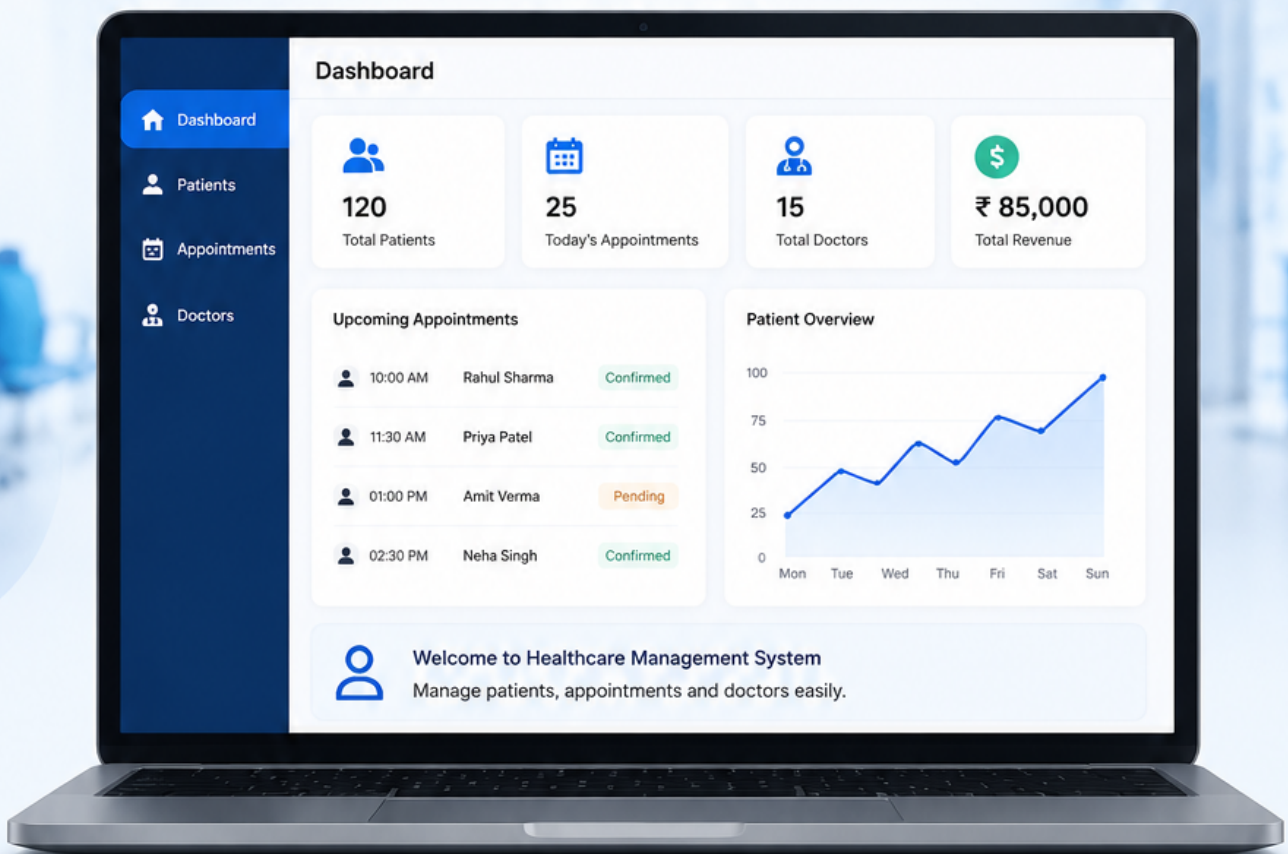




HEALTHCARE MANAGEMENT SYSTEM

Smart Healthcare. Better Management.
Better Care.



PATIENT
MANAGEMENT



APPOINTMENT
SCHEDULING



DOCTOR
MANAGEMENT



Simplify Operations. Improve Care.
All in One Healthcare Solution.

TABLE OF CONTENTS

- 1 Introduction
- 2 MySQL Database & Tables
- 3 Source Code
- 4 Table Structures
- 5 Limitations

1 Introduction

A **Healthcare Management System** is designed to simplify the interaction between doctors and patients by efficiently managing appointments. In many clinics and hospitals, scheduling appointments manually often leads to long waiting times, confusion, and miscommunication. A digital system addresses these issues by allowing patient details, doctor information, and appointment timings to be stored and managed systematically.

This system ensures that patients can easily book appointments with the right doctors, while doctors can view their schedules clearly and plan their consultations. It reduces the chances of double-booking, missed appointments, and unnecessary delays. With organised record-keeping and quick access to patient information, both doctors and patients experience a smoother and more reliable healthcare process.

Key Features

Appointment Booking

Patients can book appointments with the right specialist, reducing confusion and waiting times.

Record Management

Doctor and patient records are stored systematically for quick and reliable retrieval.

Relational Database

MySQL foreign keys link APPOINTMENT2 to PATIENT1 and DOCTOR2, enforcing data integrity.

Python CLI Interface

A menu-driven Python program provides straightforward terminal-based access to all features.

2 MySQL Database & Tables

Database Creation

The project uses a dedicated MySQL database named **HEALTHCARE**. Three relational tables are created inside it to store doctors, patients, and appointments.

```
create database HEALTHCARE;
```

Table 1 – DOCTOR2

Stores information about doctors registered in the system.

```
use HEALTHCARE;
create table DOCTOR2(
    Sl_NO          int,
    Dr_ID          char(10)    PRIMARY KEY,
    Dr_Name        varchar(30),
    Specialisation varchar(20),
    DOJ            date
);
```

Column	Data Type	Constraint	Description
Sl_NO	INT	—	Serial number
Dr_ID	CHAR(10)	PRIMARY KEY	Unique doctor identifier
Dr_Name	VARCHAR(30)	—	Full name of doctor
Specialisation	VARCHAR(20)	—	Medical speciality
DOJ	DATE	—	Date of joining

Table 2 – PATIENT1

Stores demographic and health information for each patient.

```

use HEALTHCARE;
create table PATIENT1(
    Sl_NO      int,
    Pt_Code    char(10)    PRIMARY KEY,
    Pt_Name    varchar(30),
    Health_Issue varchar(20),
    Gender     char(10),
    Age        int,
    City       varchar(15)
);

```

Column	Data Type	Constraint	Description
Sl_NO	INT	—	Serial number
Pt_Code	CHAR(10)	PRIMARY KEY	Unique patient code
Pt_Name	VARCHAR(30)	—	Full name of patient
Health_Issue	VARCHAR(20)	—	Primary health complaint
Gender	CHAR(10)	—	Gender of patient
Age	INT	—	Age in years
City	VARCHAR(15)	—	City of residence

Table 3 – APPOINTMENT2

Links patients and doctors via foreign keys, and tracks appointment details.

```

use HEALTHCARE;
create table APPOINTMENT2(
    Sl_NO      int,
    Pt_Code    char(10),
    Dr_ID      char(10),
    Appt_Time  char(10),
    Appt_Date  date,
    Appt_Status varchar(15),
    FOREIGN KEY (Pt_Code) REFERENCES PATIENT1(Pt_Code),
    FOREIGN KEY (Dr_ID)   REFERENCES DOCTOR2(Dr_ID)
);

```

Column	Data Type	Constraint	Description
Sl_NO	INT	—	Serial number
Pt_Code	CHAR(10)	FK → PATIENT1	Patient reference
Dr_ID	CHAR(10)	FK → DOCTOR2	Doctor reference

Column	Data Type	Constraint	Description
Appt_Time	CHAR(10)	—	Scheduled time
Appt_Date	DATE	—	Date of appointment
Appt_Status	VARCHAR(15)	—	Confirmed / Pending / Done

3 Source Code

Module Overview

The application is written in **Python 3** and uses the **mysql-connector-python** library to communicate with the MySQL database. It is structured as a collection of functions, each responsible for one CRUD operation, and driven by a **while-True** menu loop.

1 · Database Connection

```
import mysql.connector
con = None

def connect():
    global con
    con = mysql.connector.connect(
        host      = "localhost",
        user      = "root",
        passwd    = "mysql",
        database  = "HEALTHCARE"
    )
```

2 · Doctor Functions

```
def addDoctor():
    print("\n ADDING A NEW DOCTOR")
    Sl_NO      = int(input("Enter Serial No: "))
    Dr_ID      = input("Enter Doctor Id: ")
    Dr_Name    = input("Enter Doctor Name: ")
    Specialisation = input("Enter Doctor Specialisation: ")
    DOJ        = input("Enter Date of Joining: ")
    c1 = con.cursor()
    q = "INSERT INTO DOCTOR2 VALUES({}, '{}', '{}', '{}', '{}')".format(
        Sl_NO, Dr_ID, Dr_Name, Specialisation, DOJ)
    c1.execute(q)
    con.commit()
    print("Doctor details added successfully")

def showDoctors():
    print("\n SHOWING ALL DOCTOR DETAILS")
    c1 = con.cursor()
    c1.execute("SELECT * FROM DOCTOR2")
    rec = c1.fetchall()
    print("S.No   Dr_ID   Dr_Name   Specialisation   DOJ")
    for i in rec:
        print(i)

def delDoctor():
    print("\n DELETING DOCTOR DETAILS")
    did = input("Enter Doctor Id to delete: ")
    c1 = con.cursor()
    q = "DELETE FROM DOCTOR2 WHERE Dr_ID='{}'".format(did)
    c1.execute(q)
    con.commit()
    print("Doctor record deleted successfully")
```

3 · Patient Functions


```
def addPatient():
    print("\n ADDING A NEW PATIENT")
    Sl_NO      = int(input("Enter Serial No: "))
    Pt_Code    = input("Enter Patient Code: ")
    Pt_Name    = input("Enter Patient Name: ")
    Health_Issue = input("Enter Patient's Health Issue: ")
    Gender     = input("Enter Gender: ")
    Age        = int(input("Enter Age: "))
    City       = input("Enter City: ")
    c1 = con.cursor()
    q = "INSERT INTO PATIENT1 VALUES({}, '{}', '{}', '{}', '{}', {}, '{}')".format(
        Sl_NO, Pt_Code, Pt_Name, Health_Issue, Gender, Age, City)
    c1.execute(q)
    con.commit()
    print("Patient details added successfully")

def showPatient():
    print("\n SHOWING ALL PATIENT DETAILS")
    c1 = con.cursor()
    c1.execute("SELECT * FROM PATIENT1")
    rec = c1.fetchall()
    print("S.No  Pt_Code  Pt_Name  Health_Issue  Gender  Age  City")
    for i in rec:
        print(i)

def delPatient():
    print("\n DELETING PATIENT DETAILS")
    pcode = input("Enter Patient Code to delete: ")
    c1 = con.cursor()
    q = "DELETE FROM PATIENT1 WHERE Pt_Code='{}'.format(pcode)
    c1.execute(q)
    con.commit()
    print("Patient record deleted successfully")
```

4 · Appointment Functions

```
def apptStatus():
    print("\n SHOWING APPOINTMENT STATUS")
    apt = input("Enter Patient Code: ")
    c1 = con.cursor()
    q = "SELECT Pt_Code, Appt_Status FROM APPOINTMENT2 WHERE Pt_Code='{}'.format(apt)
    c1.execute(q)
    rec = c1.fetchall()
    for r in rec:
        print(r)

def apptDate():
    print("\n SHOWING APPOINTMENT DATE")
    apt = input("Enter Patient Code: ")
    c1 = con.cursor()
    q = "SELECT Pt_Code, Appt_Date FROM APPOINTMENT2 WHERE Pt_Code='{}'.format(apt)
    c1.execute(q)
    rec = c1.fetchall()
    for r in rec:
        print(r)
```

5 · Main Menu Loop

```
connect()
print("Connected to HEALTHCARE Database")

while True:
    print("\n -----HEALTHCARE MANAGEMENT-----")
    print("1 - Add a New Doctor")
    print("2 - Show details of all Doctors")
    print("3 - Delete Doctor record")
    print("4 - Add a New Patient")
    print("5 - Show all Patient records")
    print("6 - Delete Patient record")
    print("7 - Show Appointment Status")
    print("8 - Show Appointment Date")
    print("9 - EXIT")

    ch = int(input("Enter Your Choice : "))

    if ch == 1: addDoctor()
    elif ch == 2: showDoctors()
    elif ch == 3: delDoctor()
    elif ch == 4: addPatient()
    elif ch == 5: showPatient()
    elif ch == 6: delPatient()
    elif ch == 7: apptStatus()
    elif ch == 8: apptDate()
    elif ch == 9:
        print("EXITING..... Thank You!")
        break
    else:
        print("Invalid choice! Please try again.")
```

4 Table Structures

The following **DESC** outputs confirm column definitions, data types, and key constraints as reflected in MySQL Workbench.

DOCTOR2 — DESC output

Field	Type	Null	Key	Default	Extra
SI_NO	int	YES		NULL	
Dr_ID	char(10)	NO	PRI	NULL	
Dr_Name	varchar(30)	YES		NULL	
Specialisation	varchar(20)	YES		NULL	
DOJ	date	YES		NULL	

PATIENT1 — DESC output

Field	Type	Null	Key	Default	Extra
SI_NO	int	YES		NULL	
Pt_Code	char(10)	NO	PRI	NULL	
Pt_Name	varchar(30)	YES		NULL	
Health_Issue	varchar(20)	YES		NULL	
Gender	char(10)	YES		NULL	
Age	int	YES		NULL	
City	varchar(15)	YES		NULL	

APPOINTMENT2 — DESC output

Field	Type	Null	Key	Default	Extra
SI_NO	int	YES		NULL	
Pt_Code	char(10)	YES	MUL	NULL	
Dr_ID	char(10)	YES	MUL	NULL	

Field	Type	Null	Key	Default	Extra
Appt_Time	char(10)	YES		NULL	
Appt_Date	date	YES		NULL	
Appt_Status	varchar(15)	YES		NULL	

5 Limitations

While the system fulfils core Class XII project requirements, several areas can be improved in future iterations.

1 Data Redundancy

Without careful normalisation or constraints, the same doctor or patient information (such as names) can be duplicated across tables, leading to inconsistency and wasted storage.

2 Missing Unique Identifiers

Using only SI_No as a primary key may not suffice. Doctor IDs and patient codes should be strictly enforced as unique keys to prevent duplicate entries and maintain relational integrity.

3 Limited Information Scope

The current structure does not accommodate additional useful data such as patient contact numbers, doctor qualifications, appointment reasons, or detailed status tracking.

4 No Input Validation

No validation rules are enforced for gender or age, which may allow inconsistent or invalid data to be stored in the database.

5 Single Specialisation / Condition Field

A single field for specialisation or health issue restricts recording multiple specialities per doctor or multiple health conditions per patient.

6 No Security or Authentication

The system has no login mechanism, password protection, or access control. Any user who can run the program gains full read and write access to sensitive patient and doctor records. There is also no encryption for data stored in the database, making it vulnerable to unauthorised access.

Future Scope: This project can be extended with a Tkinter or web-based GUI, input validation, password authentication, prescription management, and multi-specialisation support to evolve into a production-ready healthcare application.